UNITÀ
01
LEZIONE
04

Le funzioni ricorsive

Chiamate, stack e risultati

Prof. Francesco Gobbi
Anno scolastico 2026/2027
I.I.S.S. Galileo Galilei – Ostiglia (MN)

Una funzione può richiamare se stessa

Una funzione è **ricorsiva** quando, durante la propria esecuzione, richiama se stessa.

Ogni chiamata affronta una **versione più piccola dello stesso problema** e usa il risultato della chiamata successiva per costruire il proprio.

Esempio: la somma dei numeri da 1 a 3, indicata con $S(3)$, si ottiene **sviluppando** la definizione $S(n) = n + S(n-1)$:

$$S(3) \rightarrow 3 + S(2) \rightarrow 3 + 2 + S(1) \rightarrow 3 + 2 + 1 + S(0) \rightarrow 3 + 2 + 1 + \mathbf{0} = 6$$

la definizione si ferma qui

Lo sviluppo si chiude con $S(0) = 0$: questo valore è noto direttamente, senza altre chiamate, e permette a tutte le somme lasciate in sospeso di completarsi.

Le tre condizioni per una ricorsione corretta

1. Caso base

Un caso semplice viene risolto **senza nuove chiamate**: per la sommatoria, $S(0) = 0$.

2. Caso ricorsivo

Il problema viene espresso mediante lo stesso problema su un dato più piccolo:
$S(n) = n + S(n-1)$.

3. Avvicinamento al caso base

Ogni chiamata riceve un argomento più vicino al caso base. Con $n \geq 0$, la chiamata con `n - 1` produce la sequenza $n,\ n-1,\ n-2, \ldots, 0$: a quel punto non vengono effettuate altre chiamate.

Il caso base deve essere **raggiungibile**. La sola presenza di un `if` non garantisce che la ricorsione termini.

La sommatoria: caso base e caso ricorsivo

Il programma calcola $S(3) = 1 + 2 + 3$ con una funzione ricorsiva. Le righe sono numerate perché gli schemi seguenti indicano **quale riga sta eseguendo** ogni chiamata.

```
1  def sommatoria(n):  # Intero n >= 0
2      if n == 0:  # Caso base
3          return 0
4      else:  # Caso ricorsivo
5          return n + sommatoria(n - 1)
6
7  def main():
8      print(sommatoria(3))  # Mostra 6
9
10 if __name__ == '__main__':
11     main()
```

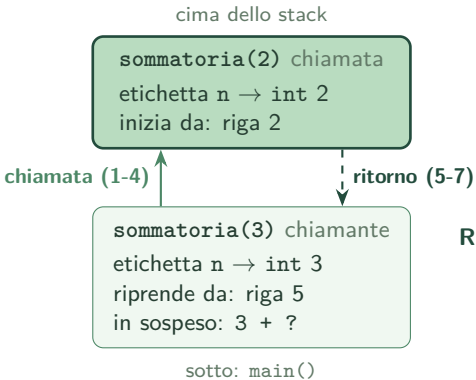
Ramo if – caso base (righe 2-3): con 0 il risultato è noto e viene restituito subito.

Ramo else – caso ricorsivo (riga 5): viene calcolata la somma fino a `n - 1`; al ritorno viene aggiunto `n`.

Il `return` della riga 5 **attende** il risultato della chiamata interna.

Ogni chiamata ha un proprio **frame**: un'etichetta `n` e un punto di ripresa.

Che cosa fa il computer a ogni chiamata



Lo stack è una **pila**: l'ultimo frame creato è il primo a terminare.

Chiamata

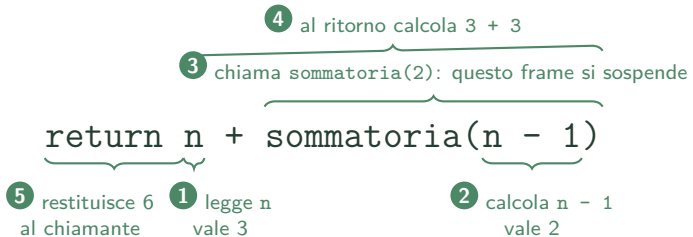
1. Viene calcolato l'argomento: `n - 1` vale 2.
2. Viene creato un **nuovo frame** in cima allo stack, con una nuova etichetta `n` associata a 2.
3. Il frame del chiamante resta sospeso e conserva il **punto di ripresa** e l'operazione da completare.
4. La funzione viene eseguita dalla prima riga, nel nuovo frame.

Ritorno

5. `return` consegna al chiamante il **riferimento al risultato**.
6. Il frame terminato viene **rimosso** dallo stack, con le sue etichette locali.
7. Il chiamante riprende dal punto salvato e completa `3 + risultato`.

L'ordine dei passaggi nella riga 5

Per eseguire `return n + sommatoria(n - 1)` nel frame con `n = 3`, Python procede in quest'ordine:



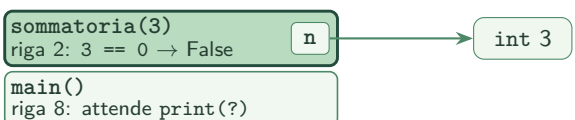
Il valore di `n` (3) viene letto **prima** della chiamata: durante l'attesa il frame conserva un'operazione incompleta, che negli schemi è scritta `3 + ?`. Il risultato della chiamata entra **direttamente nell'espressione**, senza essere assegnato a un'etichetta: la somma si completa solo nella fase di ritorno.

Traccia di sommatoria(3) – discesa 1 di 4

Codice: riga in esecuzione

```
1 def sommatoria(n):
2     if n == 0:
3         return 0
4     else:
5         return n + sommatoria(n - 1)
6
7 def main():
8     print(sommatoria(3))
```

Stack delle chiamate



Oggetti `int`

`main()` chiama `sommatoria(3)`: in cima allo stack viene creato un frame con la propria etichetta `n`, associata a 3. Il test del caso base è falso: l'esecuzione prosegue nel ramo `else`.

Verde intenso: in esecuzione Tratteggio: terminato Freccia piena: riferimento Freccia tratteggiata: valore di ritorno

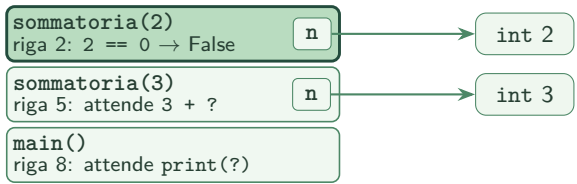
Traccia di sommatoria(3) – discesa 2 di 4

Codice: riga in esecuzione

```
1 def sommatoria(n):
2     if n == 0:
3         return 0
4     else:
5         return n + sommatoria(n - 1)
6
7 def main():
8     print(sommatoria(3))
```

Stack delle chiamate

Oggetti `int`



Nella riga 5 il frame con `n = 3` legge 3, calcola `n - 1` e chiama `sommatoria(2)`: si sospende conservando `3 + ?`. Il nuovo frame ha un'etichetta `n` **distinta**, associata a 2.

Verde intenso: in esecuzione Tratteggio: terminato Freccia piena: riferimento Freccia tratteggiata: valore di ritorno

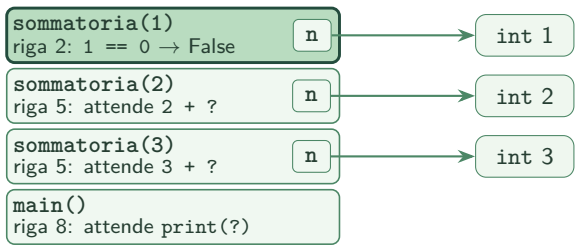
Traccia di sommatoria(3) – discesa 3 di 4

Codice: riga in esecuzione

```
1 def sommatoria(n):
2     if n == 0:
3         return 0
4     else:
5         return n + sommatoria(n - 1)
6
7 def main():
8     print(sommatoria(3))
```

Stack delle chiamate

Oggetti `int`



Lo stesso meccanismo si ripete: ora sono sospese due somme, `3 + ?` e `2 + ?`. Esistono tre etichette `n`, una per frame, ma **solo quella in cima** è usata dalla riga in esecuzione.

Verde intenso: in esecuzione Tratteggio: terminato Freccia piena: riferimento Freccia tratteggiata: valore di ritorno

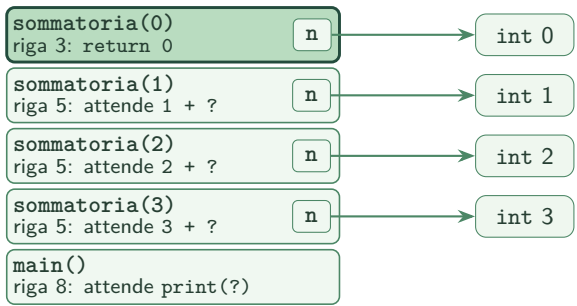
Traccia di sommatoria(3) – discesa 4 di 4: caso base

Codice: riga in esecuzione

```
1 def sommatoria(n):
2     if n == 0:
3         return 0
4     else:
5         return n + sommatoria(n - 1)
6
7 def main():
8     print(sommatoria(3))
```

Stack delle chiamate

Oggetti `int`



Con `n = 0` il test è vero: **caso base**. Lo stack ha raggiunto la massima altezza (4 frame di `sommatoria` sopra `main()`). Nessuna nuova chiamata: il frame restituisce 0.

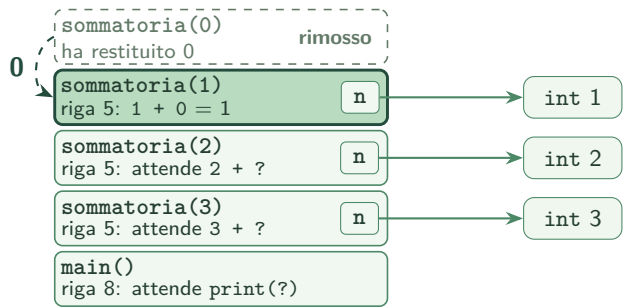
Verde intenso: in esecuzione Tratteggio: terminato Freccia piena: riferimento Freccia tratteggiata: valore di ritorno

Traccia di sommatoria(3) – risalita 1 di 4

Codice: riga in esecuzione

```
1 def sommatoria(n):
2     if n == 0:
3         return 0
4     else:
5         return n + sommatoria(n - 1)
6
7 def main():
8     print(sommatoria(3))
```

Stack delle chiamate **Oggetti `int`**



Il frame con `n = 0` termina e viene rimosso. Il frame con `n = 1` **riprende dal punto salvato**: al posto di `?` usa 0, completa `1 + 0` e restituisce 1.

Verde intenso: in esecuzione Tratteggio: terminato Freccia piena: riferimento Freccia tratteggiata: valore di ritorno

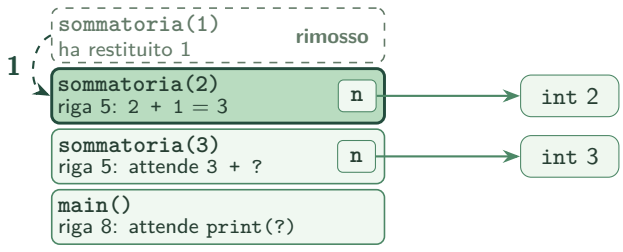
Traccia di sommatoria(3) – risalita 2 di 4

Codice: riga in esecuzione

```
1 def sommatoria(n):
2     if n == 0:
3         return 0
4     else:
5         return n + sommatoria(n - 1)
6
7 def main():
8     print(sommatoria(3))
```

Stack delle chiamate

Oggetti `int`



Il frame con `n = 1` viene rimosso. Riprende quello con `n = 2`: completa `2 + 1` e restituisce 3. Lo stack **si abbassa** di un frame a ogni ritorno.

Verde intenso: in esecuzione Tratteggio: terminato Freccia piena: riferimento Freccia tratteggiata: valore di ritorno

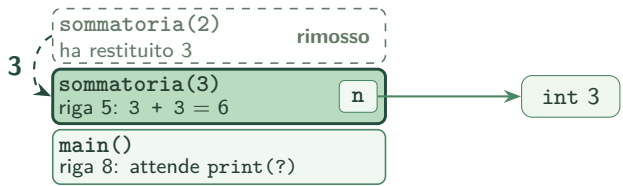
Traccia di sommatoria(3) – risalita 3 di 4

Codice: riga in esecuzione

```
1 def sommatoria(n):
2     if n == 0:
3         return 0
4     else:
5         return n + sommatoria(n - 1)
6
7 def main():
8     print(sommatoria(3))
```

Stack delle chiamate

Oggetti `int`



Riprende il primo frame creato, con `n = 3`: la sua etichetta `n` indica ancora 3, perché nessuna chiamata l'ha modificata. Completa `3 + 3` e restituisce 6.

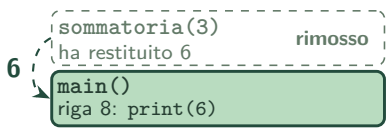
Verde intenso: in esecuzione Tratteggio: terminato Freccia piena: riferimento Freccia tratteggiata: valore di ritorno

Traccia di sommatoria(3) – risalita 4 di 4

Codice: riga in esecuzione

```
1 def sommatoria(n):
2     if n == 0:
3         return 0
4     else:
5         return n + sommatoria(n - 1)
6
7 def main():
8     print(sommatoria(3))
```

Stack delle chiamate



Oggetti `int`

Anche `sommatoria(3)` termina: nello stack resta solo `main()`, che riceve 6 e lo stampa. **Output: 6**. In totale: 4 chiamate in discesa, 4 ritorni in risalita.

Verde intenso: in esecuzione Tratteggio: terminato Freccia piena: riferimento Freccia tratteggiata: valore di ritorno

Frame, etichette e oggetti

Una nuova chiamata crea un nuovo frame, cioè un insieme distinto di etichette locali. Il codice della funzione resta uno solo: non viene copiato.

`n - 1` produce un oggetto `int`; la **nuova** etichetta `n` viene associata a quell'oggetto. L'oggetto indicato dall'etichetta `n` del chiamante **non cambia**: gli interi sono immutabili.

Il valore restituito arriva al chiamante come **riferimento a un oggetto** e viene usato subito nell'espressione: non serve assegnarlo a una variabile. Quando un frame viene rimosso, le sue etichette locali non esistono più.

Gli schemi mostrano a quale valore punta ogni etichetta, **non contano le allocazioni reali**: CPython riusa gli oggetti dei piccoli interi (da -5 a 256), quindi calcolare `n - 1` non implica sempre la creazione di un nuovo oggetto.

Il fattoriale: caso base e caso ricorsivo

Il fattoriale di n è il prodotto $n! = n \cdot (n-1) \cdots 2 \cdot 1$, con $0! = 1$. Per esempio $3! = 3 \cdot 2 \cdot 1 = 6$.

```
1  def fattoriale(n):  # Intero n >= 0
2      if n == 0:  # Caso base: 0! = 1
3          return 1
4      else:  # Caso ricorsivo
5          return n * fattoriale(n - 1)
6
7  def main():
8      print(fattoriale(3))  # Mostra 6
9
10 if __name__ == '__main__':
11     main()
```

Ramo if – caso base: $0! = 1$.

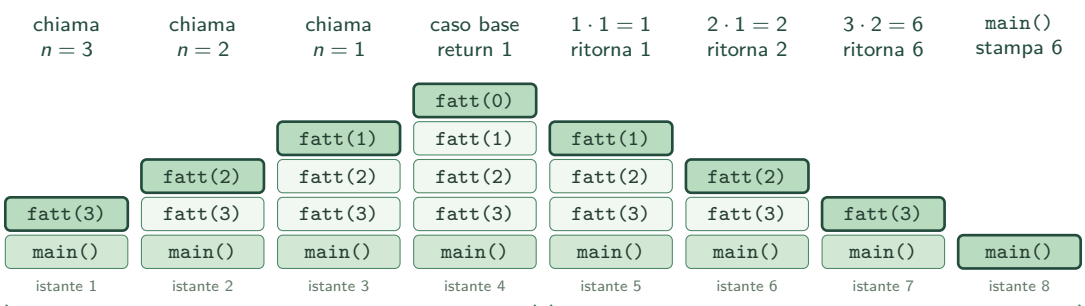
Ramo else – caso ricorsivo: $n! = n \cdot (n-1)!$.

La struttura è la stessa della sommatoria: cambiano il **valore del caso base** e l'**operazione** lasciata in sospeso.

Con `n = 3`, il frame resta in attesa con l'operazione `3 * ?`, dove `?` è il risultato di `fattoriale(2)`.

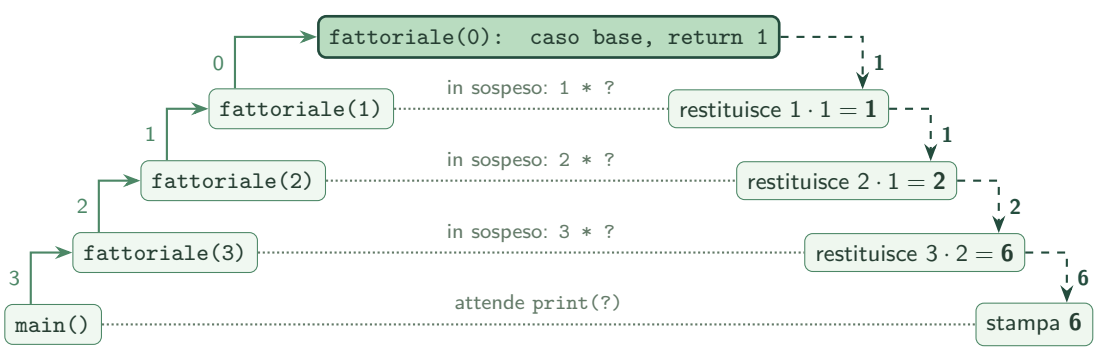
Fattoriale: lo stack nel tempo

Ogni colonna è una **fotografia dello stack** durante `fattoriale(3)`; il tempo scorre da sinistra a destra.



Il frame in cima (verde intenso) è l'unico in esecuzione. Il primo frame creato, `fatt(3)`, è l'**ultimo a terminare**.

Fattoriale: operazioni in sospeso e ritorni



Come nello stack, `main()` è in basso e il caso base in cima. Ogni riga tratteggiata è un frame **che attende**: le moltiplicazioni vengono eseguite solo al ritorno, dalla più interna (in alto) alla più esterna (in basso).

Perché il caso base deve essere corretto

Sommatoria: $S(0) = 0$, perché non ci sono addendi.

Fattoriale: $0! = 1$, l'elemento neutro del prodotto.

Con ingresso 0, la funzione restituisce subito il valore base. Con ingresso 1, esegue una sola chiamata con 0 e poi completa una sola operazione.

Se il caso base del fattoriale restituisse 0, la risalita moltiplicherebbe tutto per 0:
$3 \cdot 2 \cdot 1 \cdot \mathbf{0} = 0$.

Con un ingresso negativo, il passo `n - 1` si allontana da 0. Le funzioni presentate richiedono quindi **interi non negativi**.

La correttezza dipende sia dal **dominio ammesso** sia dal **passo che riduce il problema**. Gli esempi assumono che la precondizione $n \geq 0$ sia rispettata.

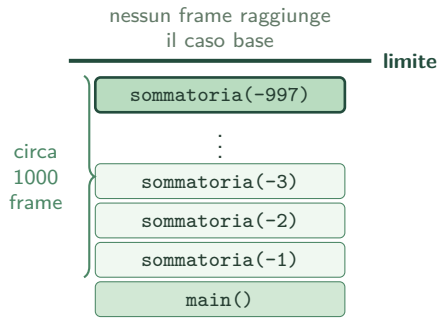
Ricorsione senza fine: lo stack si riempie

Con `sommatoria(-1)` il test `n == 0` non diventa mai vero: gli argomenti sono $-1, -2, -3, \ldots$ e si allontanano dal caso base.

Ogni chiamata aggiunge un frame. Python impone un **limite alla profondità** (di norma 1000): superato il limite, il programma si interrompe. Il limite **non è un modo per terminare**: la terminazione deve derivare dal caso base.

ERRORE

```
Traceback (most recent call last):
  ...
RecursionError: maximum recursion
depth exceeded
```



Gli errori da riconoscere

- **Chiamata con lo stesso dato:** `sommatoria(n)` al posto di `sommatoria(n - 1)`; l'argomento non si avvicina al caso base e la ricorsione non termina.
- **Risultato non restituito:** con `print(0)` al posto di `return 0` il valore viene mostrato, ma la chiamata restituisce `None`; nel chiamante `1 + None` provoca `TypeError`.
- **Caso base errato:** nel fattoriale, restituire 0 annulla tutti i prodotti della risalita.
- **Ramo else senza return:** la chiamata ricorsiva viene eseguita, ma il suo risultato va perso e la funzione restituisce `None`.
- **Troppe chiamate annidate:** anche con un caso base corretto, un ingresso molto grande può superare il limite di profondità e provocare `RecursionError`.

Ricorsione e iterazione a confronto

	Iterazione	**Ricorsione**
Ripetizione	Un ciclo ripete il corpo.	Una funzione richiama se stessa.
Stato del calcolo	Contatore e accumulatore vengono aggiornati.	Ogni frame conserva la propria etichetta `n` e l'operazione in sospeso.
Arresto	La condizione diventa falsa o i valori terminano.	Viene raggiunto il caso base.
Stack delle chiamate	Un solo frame per tutta l'esecuzione.	Un frame in più per ogni chiamata annidata: $n + 1$ frame per `sommatoria(n)`.

Per sommatoria e fattoriale entrambe le forme eseguono un numero di passi proporzionale a n; la versione ricorsiva occupa anche $n + 1$ frame.

Problema 1: il numero di caselle

Problema. Una figura è composta da n righe: la prima contiene 1 casella, la seconda 2 e così via fino alla riga n. **Creare una funzione ricorsiva** `caselle(n)` che restituisca il numero totale di caselle, per un intero $n \geq 0$. Una figura senza righe contiene 0 caselle.

```
def caselle(n):  # Numero di righe
    if n == 0:  # Caso base
        return 0  # Figura vuota
    else:  # Caso ricorsivo
        # Ultima riga + figura precedente
        return n + caselle(n - 1)

def main():  # Programma principale
    print(caselle(4))  # Mostra 10

if __name__ == '__main__':
    main()  # Avvio
```



prime 3 righe:
caselle(3) = 6
ultima riga: 4

Perché funziona: la figura con n righe contiene le n caselle dell'ultima riga più quelle delle prime $n-1$ righe.

Argomenti: $4 \to 3 \to 2 \to 1 \to 0$.
Ritorni: $0 \to 1 \to 3 \to 6 \to 10$.

Problema 2: una potenza intera

Problema. Creare una funzione ricorsiva `potenza(b, e)` che calcoli b^e, con base intera $b > 0$ ed esponente intero $e \geq 0$, usando moltiplicazioni e **senza l'operatore** `**`. Per esponente 0, il risultato è 1.

```
def potenza(b, e):  # Base ed esponente
    if e == 0:  # Caso base
        return 1  # b elevato a zero
    else:  # Caso ricorsivo
        # La base resta uguale; e diminuisce
        return b * potenza(b, e - 1)

def main():  # Programma principale
    print(potenza(2, 3))  # Mostra 8

if __name__ == '__main__':
    main()  # Avvio
```

Perché funziona: $b^e = b \cdot b^{e-1}$. La base resta uguale; è e ad avvicinarsi al caso base. Con `potenza(5, 0)` si ottiene subito 1.

`potenza(2, 3)` $2 \cdot 4 = \mathbf{8}$

`potenza(2, 2)` $2 \cdot 2 = 4$

`potenza(2, 1)` $2 \cdot 1 = 2$

`potenza(2, 0)` caso base: 1

Esercizi

1. Traccia della sommatoria

Disegnare lo stack delle chiamate di `sommatoria(2)` passo per passo: per ogni frame indicare il valore di `n`, la **riga in esecuzione**, l'**operazione in sospeso** e il **valore restituito**.

2. Traccia del fattoriale

Ricostruire discesa e risalita di `fattoriale(4)` con uno schema come quello delle operazioni in sospeso e **spiegare** perché il frame con `n = 4` è l'ultimo a terminare.

3. Somma dei quadrati

Creare una funzione ricorsiva `somma_quadrati(n)` che calcoli $1^2 + 2^2 + \cdots + n^2$, per $n \geq 0$, con il **caso base** nel ramo `if` e il **caso ricorsivo** nel ramo `else`.

Casi di prova per il terzo esercizio: con 0 si ottiene 0; con 1 si ottiene 1; con 3 si ottiene 14 $(1 + 4 + 9)$.